

**MASTER1 INTELLIGENCE ARTIFICIELLE
ET INTERNET DES OBJETS**

Internet of Things - IoT

Korota Arsène COULIBALY

ENSAM de Casablanca, Maroc



Année universitaire 2024-2025

Programme

I. Introduction à l'IoT

II. Les systèmes embarqués dans l'IoT

III. Les protocoles de communications

IV. Plateformes IoT et analyses des données collectées

Objectifs de ce cours

Objectif principal :

- Être capable de concevoir un système IoT

Objectifs pédagogiques:

- Décrire un système IoT;
- Décrire les systèmes embarqués;
- Comprendre les contraintes des systèmes embarqués vis-à-vis de l'IoT;
- Balayer l'architecture matérielle et logicielle des systèmes embarqués;
- Décrire et choisir le protocole de communication sans fil adapté pour son application IoT

Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

Le problème de la collecte des récipients en verre en milieu urbain : un spectacle pas si inhabituel ...

La solution : assurer la collecte des des récipients en verre AVANT que la poubelle ne soit pleine.

Les moyens : équiper la poubelle d'un capteur de niveau de remplissage qui alerte automatiquement le centre de collecte de ce qu'elle est presque pleine, et que donc, il faut la vider.

→ Internet des Objets



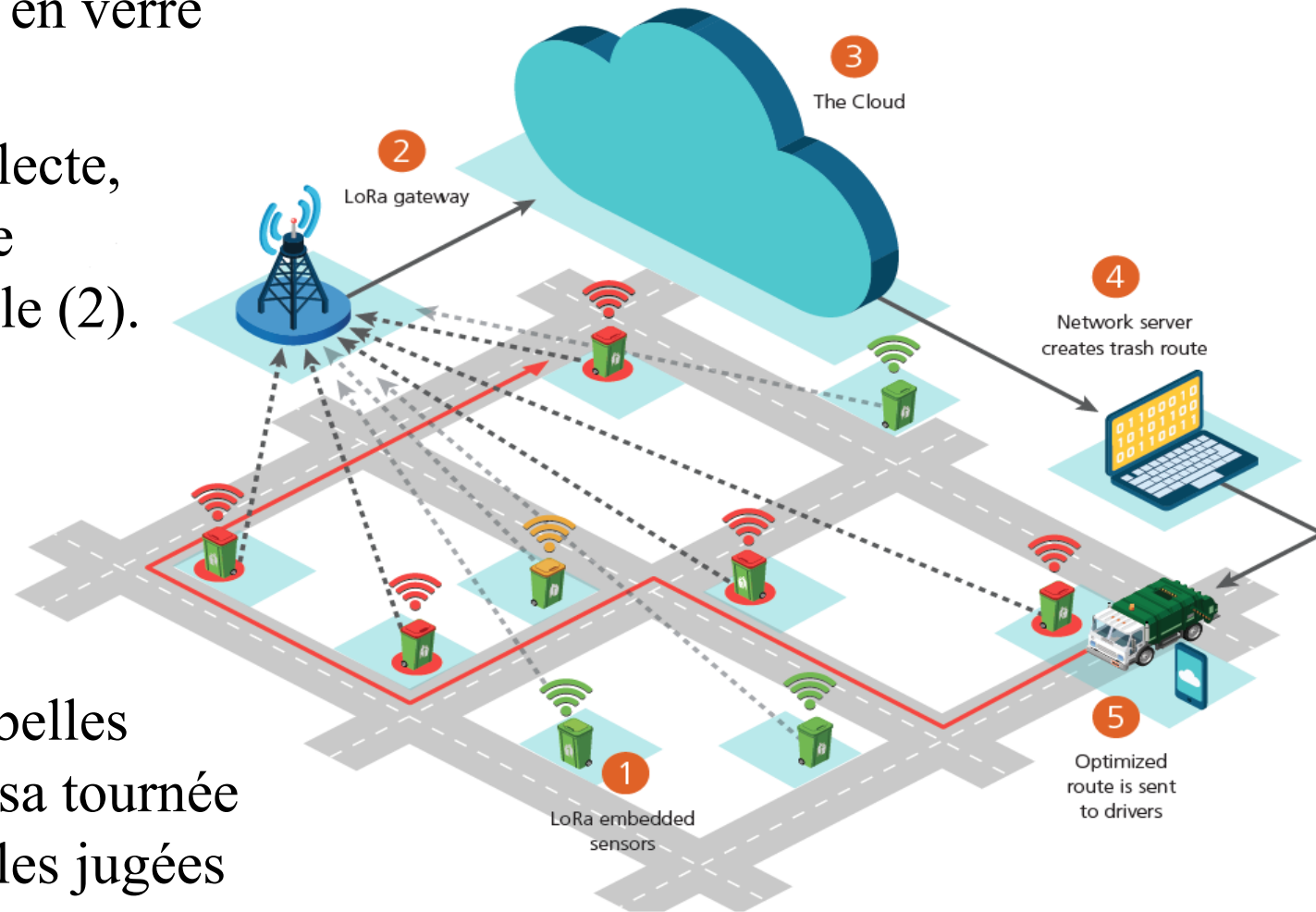
Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

Optimisation de la collecte des récipients en verre

Tous les jours, avant que ne débute la collecte, les poubelles (1) transmettent leur taux de remplissage (de 0 à 100%) à une passerelle (2).

Les informations collectées sont ensuite transmises à un serveur et mises à disposition sur le « cloud » (3).

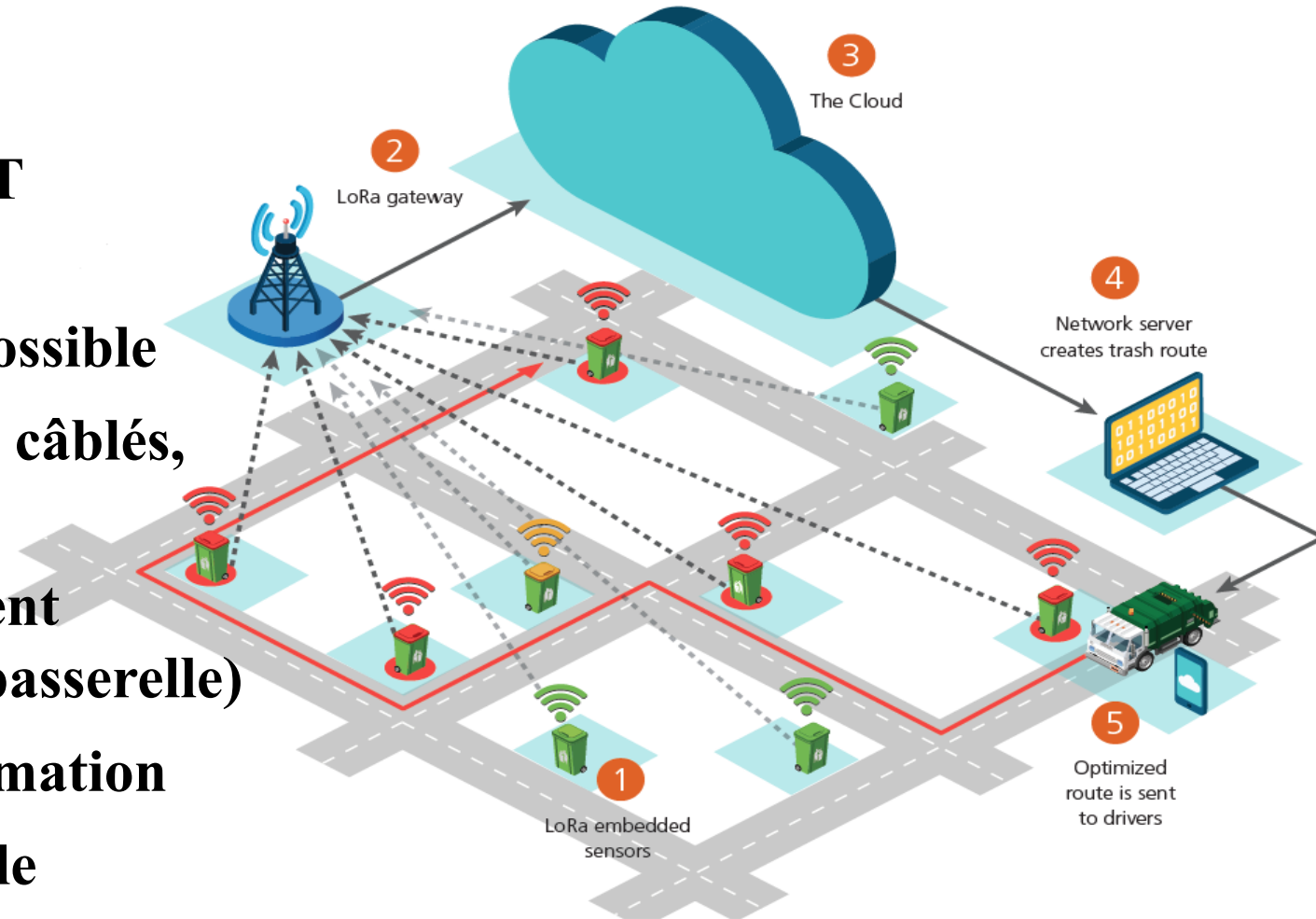
L'opérateur chargé de la collecte des poubelles récupère les informations (4) et optimise sa tournée (5) de façon à ne collecter que les poubelles jugées suffisamment pleines.



Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

Les principales caractéristiques de l'IoT

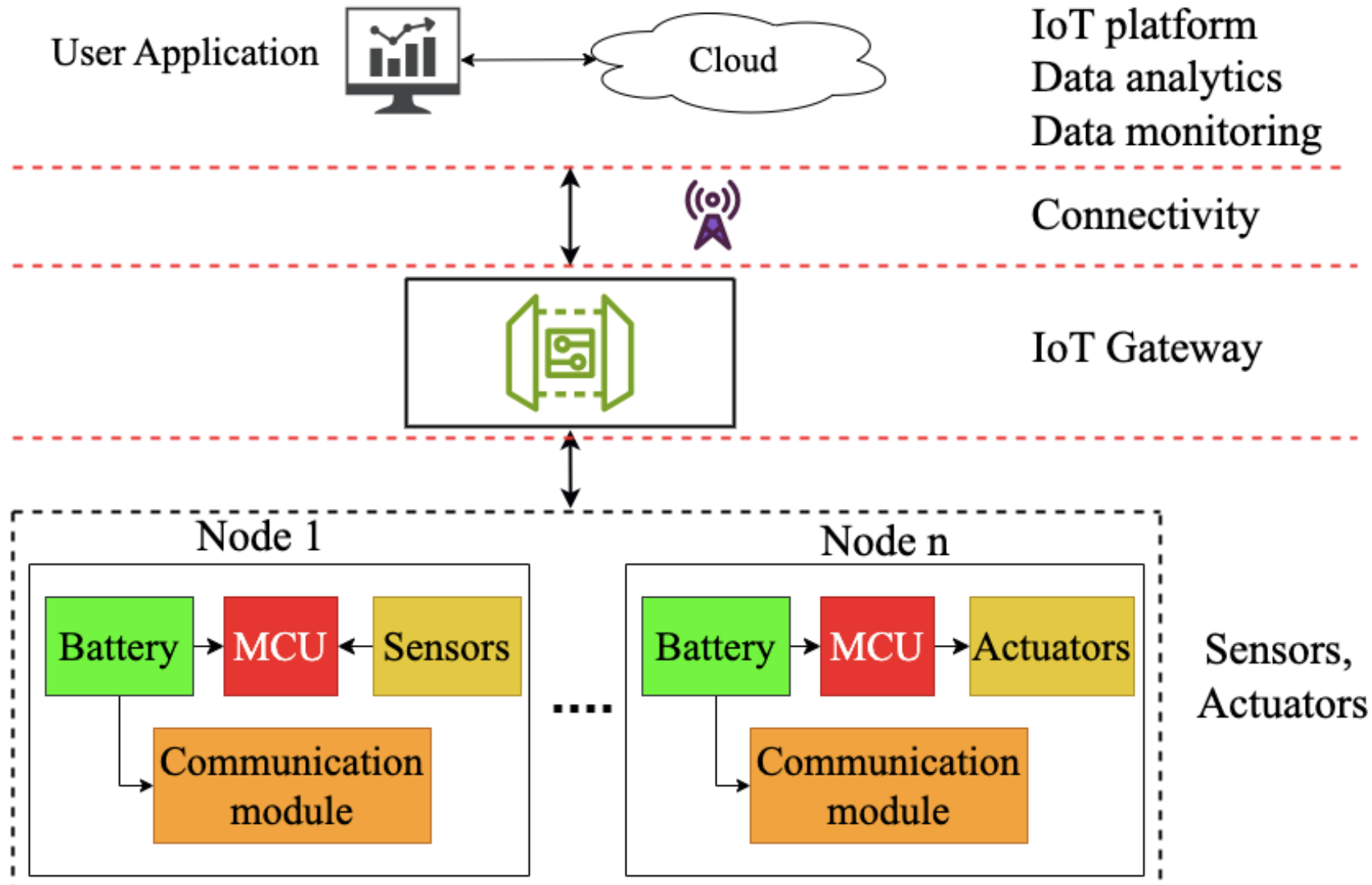
- Une couverture radio la plus large possible
- Une interopérabilité avec les réseaux câblés, sans fil et cellulaires existants
- Une transmission radio essentiellement unidirectionnelle (de l'objet vers la passerelle)
- Une technologie radio faible consommation
- Une transmission de données de faible volume, à bas débit et discontinue



Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

L'IoT est un système dans lequel des objets appelés Nœuds (Node) ou End Devices collectent des données et les injectent directement sur le réseau internet ou via une passerelle (gateway). Ces objets sont capables de s'échanger des données.

Introduction à L'Internet des Objets IdO (IoT – Internet of Things)



SSID: ESP32-Access-Point
Password: 123456789
IP Address: 192.168.4.1

Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

Caractéristiques principales d'un ED IoT

- 1. Les systèmes embarqués dans l'ED IoT**
- 2. Les protocoles de communication**

Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

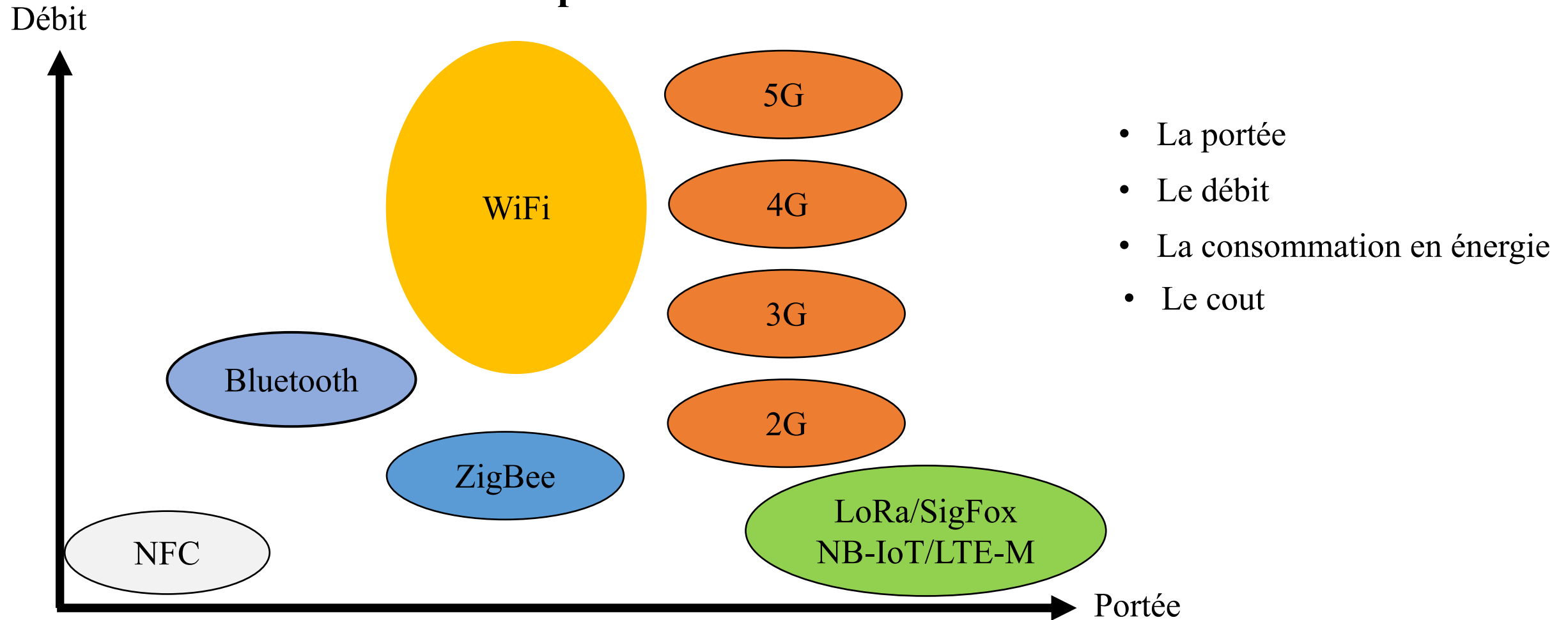
1. Les systèmes embarqués dans l'ED IoT

Les systèmes embarqués utilisés dans l'IoT possèdent les caractéristiques suivantes :

- ✓ Faible consommation énergétique
- ✓ Faible puissance de calcul
- ✓ Petite taille
- ✓ Faible coût

Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

2. Les protocoles de communications sans fils



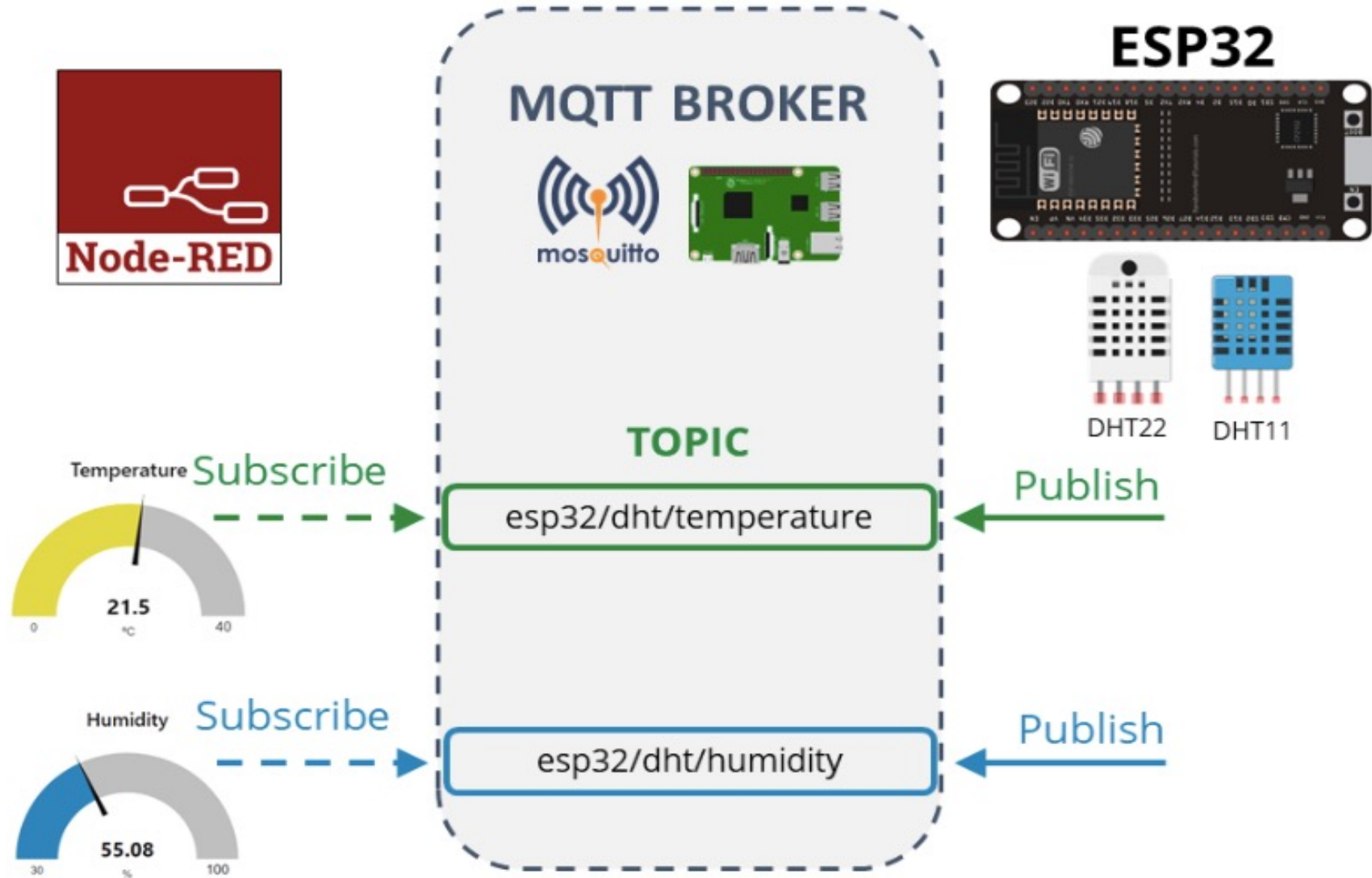
Introduction à L'Internet des Objets IdO (IoT – Internet of Things)

Soit le systèmes IoT (diapo suivante) permettant de collecter la température et l'humidité et les envoie sur un serveur mqtt. Les données sont ensuite affichés sous forme d'un dashboard à l'aide de Node-RED.

Travail à faire :

- ✓ Identifier l'ED (MCU, capteurs, actionneurs, protocole de communication utilisé.
- ✓ Les données transitent-elle via une passerelle ou non ? Si oui identifiez la passerelle
- ✓ Identifiez la plateforme IoT utilisée

Introduction à L'Internet des Objets IdO (IoT – Internet of Things)



Les systèmes embarqués

1. Les ordinateurs

Les ordinateurs de bureau :

➤ PC



➤ PC portables



➤ Serveurs

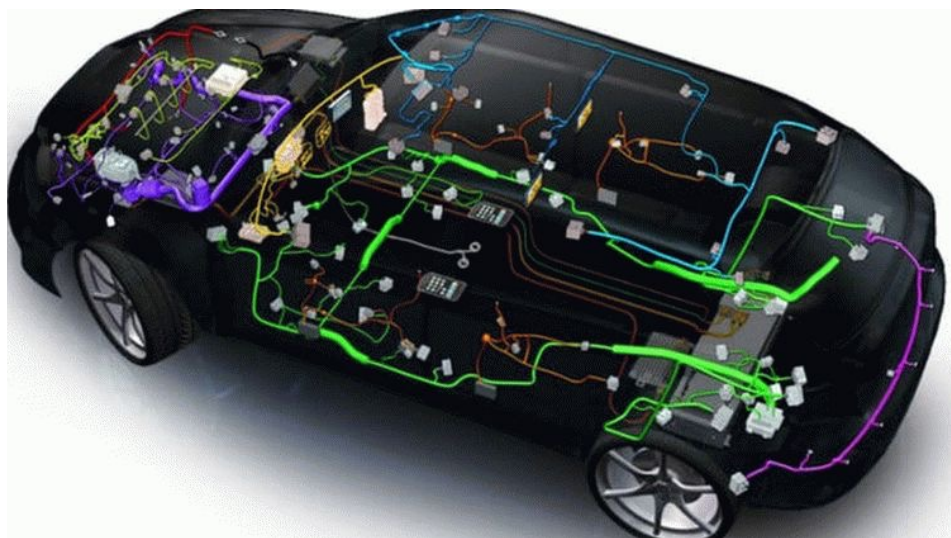


Un autre type d'ordinateurs :

- Téléphones portables
- Appareils photos numérique
- Consoles de jeux
- Imprimantes
- Systèmes de commandes et de capteurs sans fil

Les systèmes embarqués

2. Définition



Les systèmes embarqués

2. Définition

Un système embarqué peut être défini comme un système électronique et informatique autonome, qui est dédié à une tâche bien précise.

- Un système numérique
- Utilise généralement un processeur
- Exécute un logiciel dédié enfoui dans le matériel
- Ne possède pas d'entrées sorties standards et classiques

Les systèmes embarqués

2. Définition

Vocabulaire

- **Processeur** : composant qui exécute les instructions d'un programme
- **Micro-processeur** : processeur se présentant sous la forme d'un circuit intégré
- **Micro-contrôleur** : circuit intégré unique intégrant un processeur, de la mémoire et un ensemble de périphériques
- **System sur puce** (System on a chip, SoC) : définition similaire à celle d'un micro-contrôleur. La différence, floue, se fait souvent sur la taille et le nombre de composants intégrés
- **GPIO** (General-Purpose Input/Output) : patte (pin) d'un circuit intégré ou connecteur d'une carte électronique qui peut être utilisé comme une entrée ou une sortie numérique (configurable dynamiquement). Plus largement désigne le périphérique qui va permettre de contrôler ces entrées-sorties

Les systèmes embarqués

2. Définition

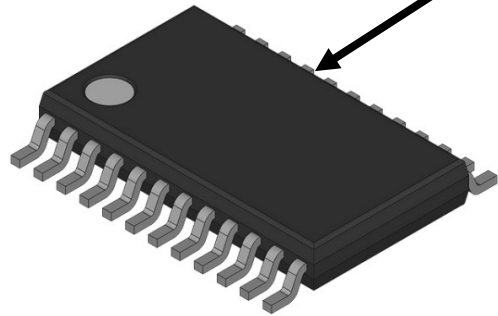
Un système embarqué doit répondre aux contraintes suivantes :

- ✓ Contrainte de mémoire
- ✓ Contrainte temporelle
- ✓ Contrainte de puissance de calcul
- ✓ Contrainte d'autonomie énergétique

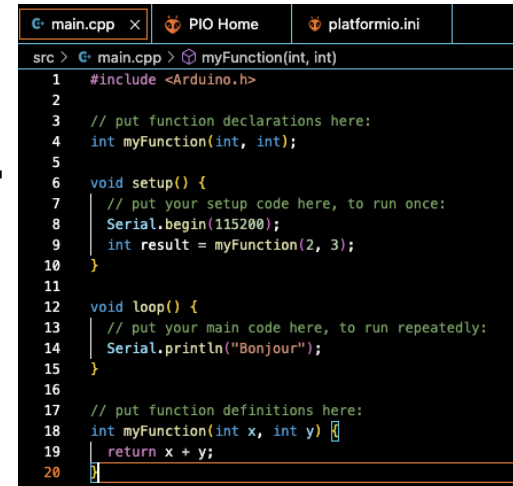
Les systèmes embarqués

2. Définition

Système embarqué = partie matériel + partie logiciel



Electronique



```
src > main.cpp > myFunction(int, int)
1 #include <Arduino.h>
2
3 // put function declarations here:
4 int myFunction(int, int);
5
6 void setup() {
7   // put your setup code here, to run once:
8   Serial.begin(115200);
9   int result = myFunction(2, 3);
10 }
11
12 void loop() {
13   // put your main code here, to run repeatedly:
14   Serial.println("Bonjour");
15 }
16
17 // put function definitions here:
18 int myFunction(int x, int y) {
19   return x + y;
20 }
```

Programme

+ RTOS

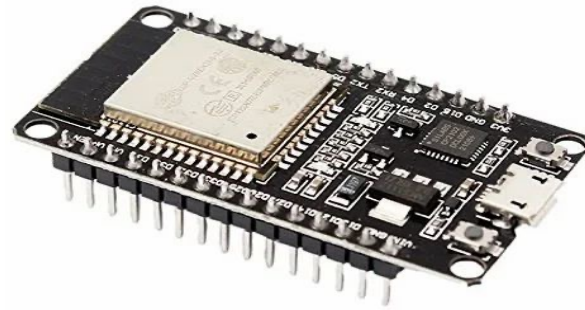
Les systèmes embarqués

3. Architecture matérielle

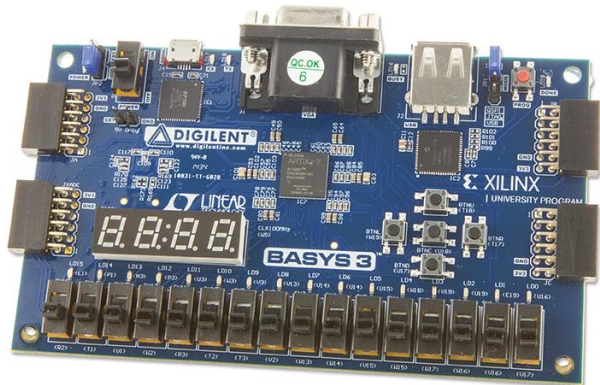
La partie matériel d'un système embarqué représente la partie où le logiciel (programme utilisateur) est embarqué. Cela pourrait être :



Un microcontrôleur



Une carte à base de microcontrôleur



Une carte à base de FPGA



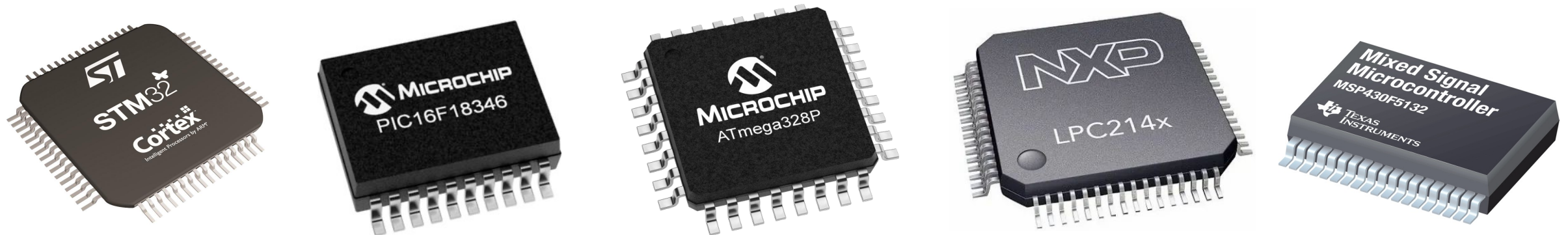
Une carte à base de DSP

Les systèmes embarqués

3. Architecture matérielle : MCU

Un microcontrôleur est un circuit intégré (un micro-ordinateur) rassemblant dans un même boîtier TOUTES les fonctions électroniques d'un système:

- Un CPU
- Une mémoire de donnée (ROM ou EEPROM)
- Des interfaces d'entrées sorties
- Des convertisseurs analogiques numériques ou numériques analogiques
- Une horloge



Les systèmes embarqués

3. Architecture matérielle : ESP32

Qu'est-ce qu'un ESP32 ? L'ESP32 est une série de microcontrôleurs à puce développée par Espressif.

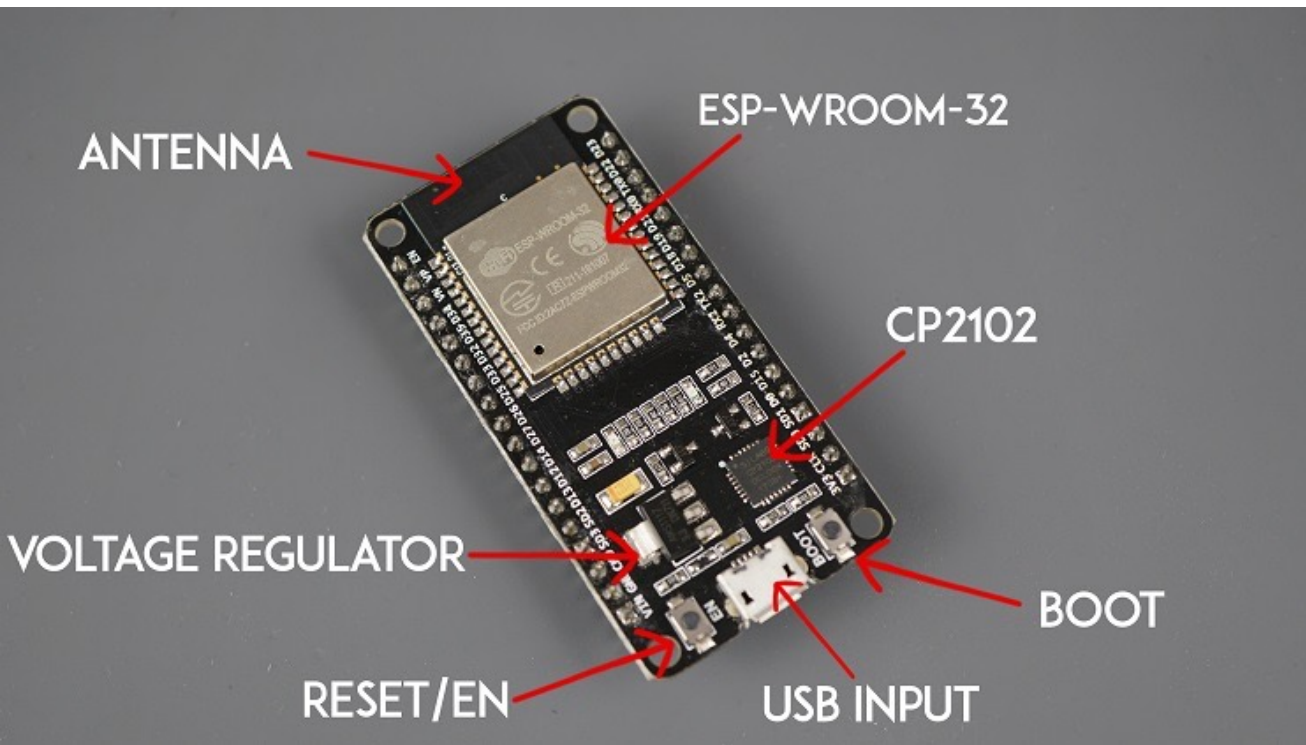


Pourquoi l'ESP32?

- ✓ Faible coût : 150 DH
- ✓ Faible consommation
- ✓ WiFi et Bluetooth (BLE y compris)
- ✓ Double cœur : livrés avec 2 microprocesseurs Xtensa 32 bits LX6 : cœur 0 et cœur 1
- ✓ Interfaces d'E/S : ADC, DAC, SPI, I2C, UART, PWM
- ✓ Compatible avec MicroPython
- ✓ Compatible avec le langage de programmation Arduino

Les systèmes embarqués

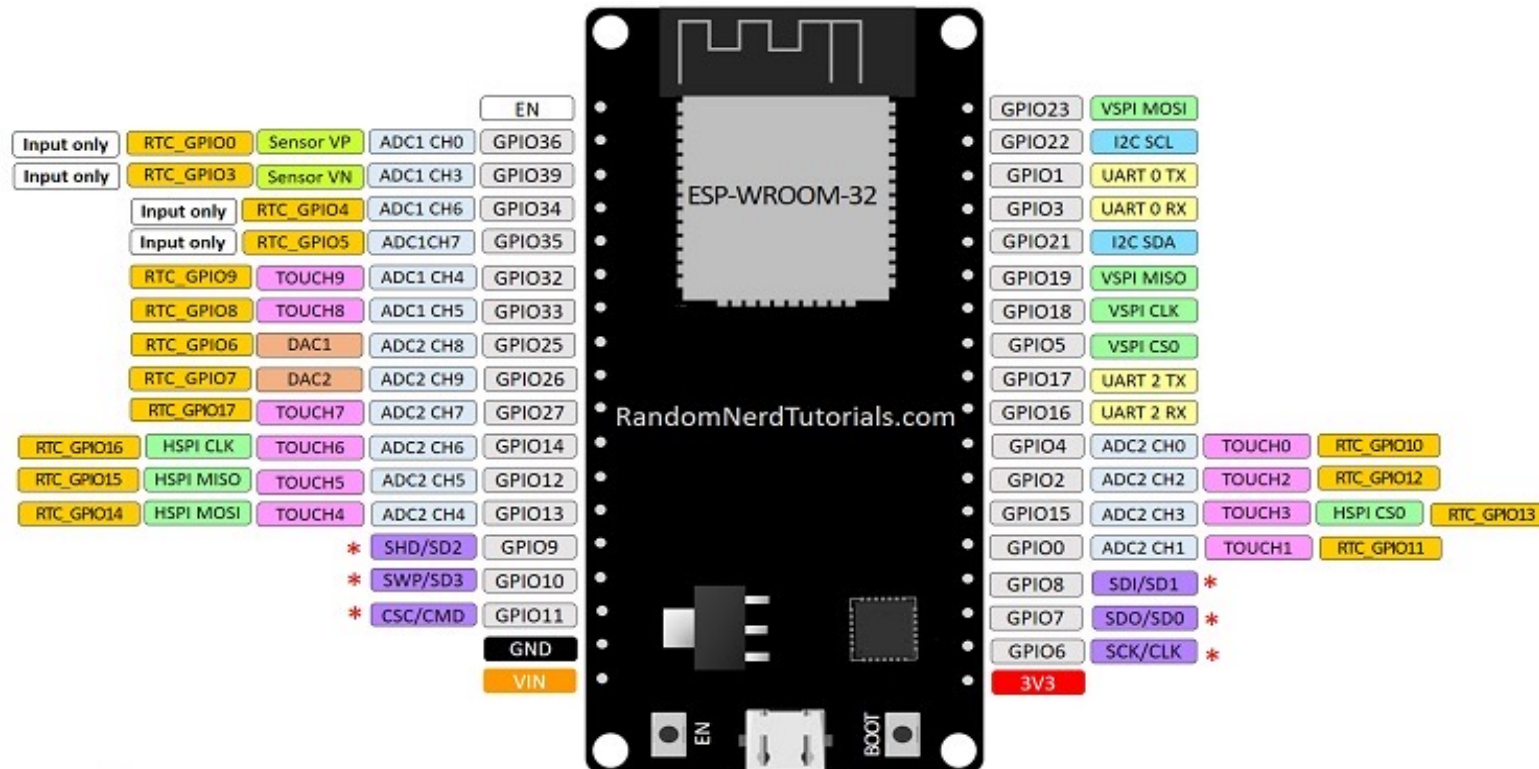
3. Architecture matérielle : ESP32



Les systèmes embarqués

3. Architecture matérielle : ESP32

ESP32 DEVKIT V1 – DOIT version with 36 GPIOs



* Pins SCK/CLK, SDO/SD0, SDI/SD1, SHD/SD2, SWP/SD3 and SCS/CMD, namely, GPIO6 to GPIO11 are connected to the integrated SPI flash integrated on ESP-WROOM-32 and are not recommended for other uses.

Les systèmes embarqués

3. Architecture matérielle : ESP32

Comment programmer l'ESP32?

- ✓ Espressif IDF (IoT Development Framework)
- ✓ Arduino C/C++ en utilisant le cœur Arduino pour l'ESP32
- ✓ MicroPython

Dans ce cours nous allons utiliser Arduino C/C++ avec PlatformIO

Les systèmes embarqués

3. Architecture matérielle : ESP32

Configuration de l'environnement de développement

Télécharger et installer



Visual Studio Code

Installer l'extension PlatformIO IDE

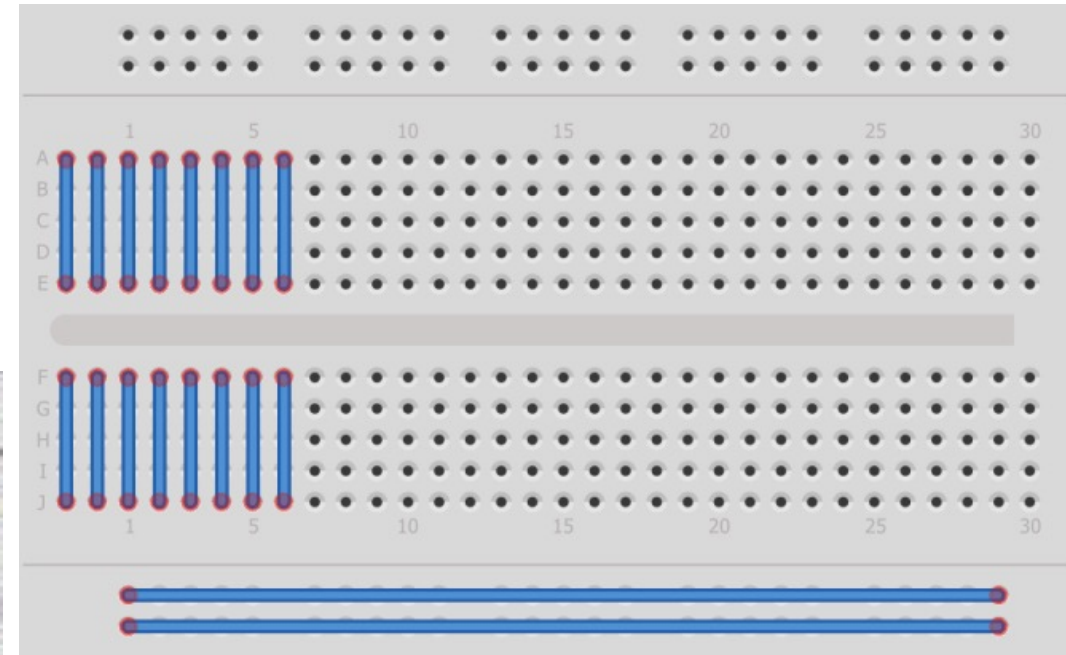


Les systèmes embarqués

3. Architecture matérielle : ESP32

Matériels nécessaires

Breadboard : Permet de relier les composants sans avoir à les souder, ce qui permet de faire des tests très facilement. Il est composé de trous permettant de connecter des composants et de les relier entre eux afin de réaliser des montages à tester.



Les systèmes embarqués

3. Architecture matérielle : ESP32

Introduction langage Arduino

C'est un programme C++ (partie cachée)

Si

```
#include <Arduino.h>

.....

int main(void)
{
    init();
    initVariant();

    .....

    setup();

    for (;;) {
        loop();
        if (serialEventRun) serialEventRun();
    }
    return 0;
}
```

Structure du sketch Arduino (partie visible)

```
void setup() {
    // put your setup code here, to run once:
}

void loop() {
    // put your main code here, to run repeatedly:
}
```

Les systèmes embarqués

3. Architecture matérielle : ESP32

Introduction langage Arduino : variables

- Les entiers
 - Données codées sur 8 bits :
 - `char` ou `int8_t` : valeurs de -128 à +127
 - `unsigned char` ou `uint8_t` : valeurs de 0 à 255
 - Données codées sur 16 bits :
 - `int` ou `int16_t` : valeurs de -32768 à 32767
 - `unsigned int` ou `uint16_t` : valeurs de 0 à 65535
 - Données sur 32 bits :
 - `long` ou `int32_t` : valeurs de -2^{31} à $2^{31} - 1$
 - `unsigned long` ou `uint32_t` : valeurs de 0 à $2^{32} - 1$
- Les réels
 - simple précision (32 bits) : `float` de $\pm 3,4 \cdot 10^{-38}$ à $\pm 3,4 \cdot 10^{38}$
 - double précision (64 bits) : `double` de $\pm 1,7 \cdot 10^{-308}$ à $\pm 1,7 \cdot 10^{308}$

Les systèmes embarqués

3. Architecture matérielle : ESP32

Introduction langage Arduino : variables

Forme générale ([.....] pour ce qui est facultatif) :

```
type nom[=valeur];
```

```
type nom1[=valeur1], nom2[=valeur2], ...;
```

Portée des variables

- Globale :

- Déclarées en dehors de toutes fonctions : `setup()`, `loop()`, fonctions de l'utilisateur
- Visibles dans toutes les fonctions

Exemple :

```
int i = 0, j = 2;
```

```
uint16_t indice;
```

```
float a;
```

```
float b = 1e5, c=0.01;
```

```
char ch1 = 'a';
```

```
char ch2;
```

Les systèmes embarqués

3. Architecture matérielle : ESP32

Introduction langage Arduino : Calculs

Opérateur d'affectation : =

Opérateurs arithmétiques

- Usuels : +, -, /, *
- Modulo : % = reste de la division entière

Incrémentation et décrémentation : ++ et --

- i++; ⇔ i=i+1;

Affectation avec calcul : +=, -=, /=, *=

- i+=3; ⇔ i=i+3;

Opérateurs logiques bit à bit

- & = ET bit à bit
- | = OU bit à bit
- ^ = OU exclusif bit à bit

Exemples :

```
uint8_t a = 0b11001100;
```

```
uint8_t b = 0b01001001;
```

```
uint8_t ou, et, ouex;
```

```
ou = a|b; // donne ou vaut 0b11001101
```

```
et = a&b; // donne et vaut 0b01001000
```

```
ouex = a^b; // donne ouex vaut 0b10000101
```

```
Serial.print("ou = ");
```

```
Serial.println(ou, BIN);
```

Les systèmes embarqués

3. Architecture matérielle : ESP32

**Introduction langage Arduino : Faire clignoter une LED
brancher sur un GPIO de l'ESP32**

- Deux fonctions natives

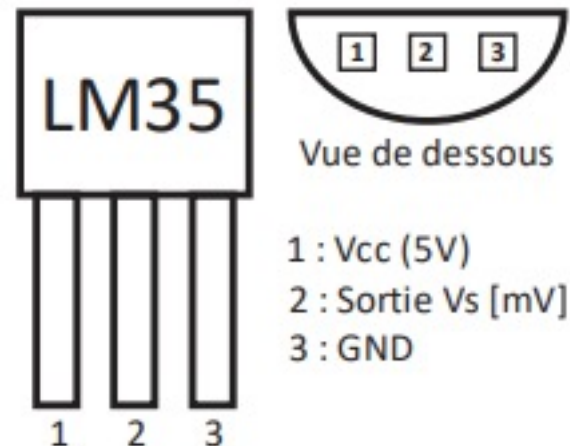
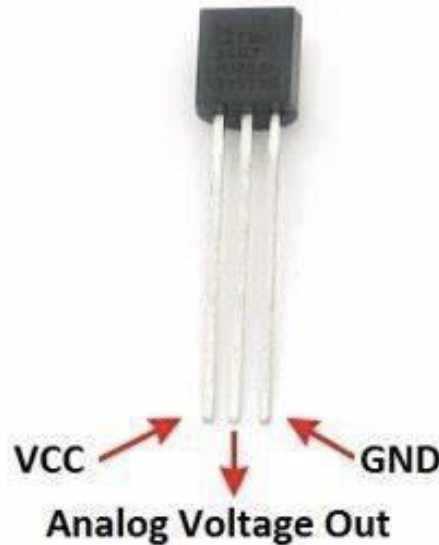
```
// Configurer une broche en sortie  
pinMode(numero_de_broche, OUTPUT);  
// Sortir +VCC sur la broche  
digitalWrite(numero_de_broche, HIGH);  
// Sortie 0V sur la broche  
digitalWrite(numero_de_broche, LOW);
```

Les systèmes embarqués

3. Architecture matérielle : ESP32

Introduction langage Arduino : Lire les données d'un capteur

Le capteur de température LM35 est un capteur analogique de température fabriqué par Texas Instruments. Il est extrêmement populaire en électronique, car précis, peu coûteux et très simple d'utilisation.



Les systèmes embarqués

3. Architecture matérielle : ESP32

Introduction langage Arduino : Lire les données d'un capteur

Le capteur de température LM35 est capable de mesurer des températures allant de -55°C à $+150^{\circ}\text{C}$

La sortie analogique du capteur est proportionnelle à la température tel que :

$$T = V_s / 10$$

Avec :

T : la Température en degré Celsius [$^{\circ}\text{C}$]

Vs : Tension de sortie en [mV]

Remarque : Il suffit de mesurer la tension en sortie du capteur pour en déduire la température. Chaque degré Celsius correspond à une tension de +10mV.